Other filters can apply to outgoing responses after they are received from the downstream application and before sending them back to the client (*post-filters*). They can be used for:

- Setting security headers
- Manipulating the response body to remove sensitive information

Spring Cloud Gateway comes bundled with many filters that you can use to perform different actions, including adding headers to a request, configuring a circuit breaker, saving the web session, retrying the request on failure, or activating a rate limiter.

In the previous chapter, you learned how to use the retry pattern to improve application resilience. You'll now learn how to apply it as a default filter for all `GET` requests going through the routes defined in the gateway.

USING THE RETRY FILTER

You can define default filters in the application.yml file located under src/main/ resources. One of the filters provided by Spring Cloud Gateway is the `Retry` filter. The configuration is similar to what we did in chapter 8.

Let's define a maximum of three retry attempts for all `GET` requests whenever the error is in the `5xx` range (`SERVER_ERROR`). We don't want to retry requests when the error is in the `4xx` range. For example, if the result is a `404` response, it doesn't make sense to retry the request. We can also list the exceptions for which a retry should be attempted, such as `IOException` and `TimeoutException`.

By now, you know that you shouldn't keep retrying requests one after the other. You should use a backoff strategy instead. By default, the delay is